

19

Cyber Security  
Codec Technologies India  
Internship 2026

# Insider Threat Detection

User Behaviour Analytics: stats + ML + rules

## AT A GLANCE

25,204

Events

60 / 30

Users / days

1.00

Precision@4

1.000

ROC-AUC

4 / 4 top-4

Insiders caught

9 / 9

Tests re-run

synthetic (CERT-like)

Data

## VERDICT

TESTS 9/9

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

Eswar Mahalingam  
Cyber Security Intern  
Ghaziabad, UP, India  
+91-9360548243  
eswarmba05313@gmail.com  
linkedin.com/in/eswar-mahalingam

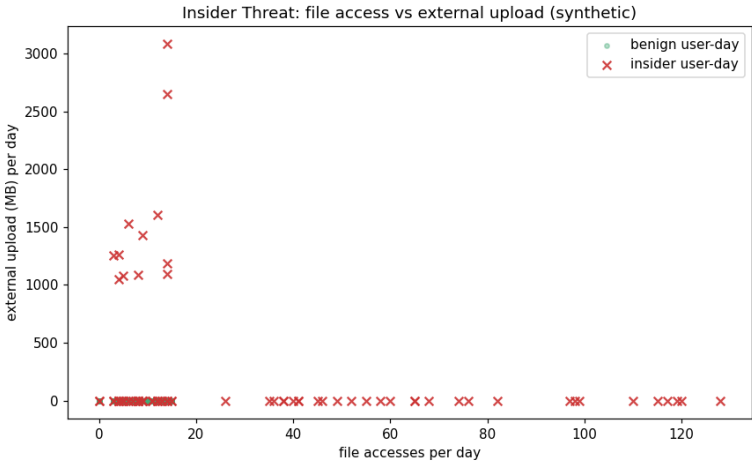
## Objective & brief mapping

The brief asked for insider-threat detection. I built a User Behaviour Analytics pipeline that engineers per-user/day features (logon timing, file access, USB, uploads, external email), then blends statistical baselines, unsupervised ML and a rules layer into one risk score that ranks the most anomalous users.

## Architecture

generate\_data.py writes CERT-inspired synthetic logs with four planted insiders as ground truth. features.py builds per-user/day rows; detect.py computes self- and peer-group z-scores, an IsolationForest anomaly rank, a rules layer, and a weighted risk score aggregated per user.

```
events -> features (per user/day)
(a) self + peer z-scores (b) IsolationForest (200 trees)
(c) rules (d) risk = 0.45*rules + 0.35*IF + 0.20*stat
-> risk_scores.csv + report + scatter + ranking
```



File access vs upload, insiders marked (results/anomaly\_scatter.png).

## Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python main.py # generate data, detect, write results/ (~9 s)
python3 unittest-style runner over tests/ # 9 tests
```

The artefacts each figure and table above are drawn from:

- risk\_scores.csv — ranked users + reasons
- top\_insiders\_report.md — named insiders + why
- anomaly\_scatter.png / feature\_risk\_ranking.png — charts

## Method & implementation

Each feature is z-scored against the user's own history and their department peer group, so "normal for this person/team" is learned per entity. The rules layer encodes analyst knowledge (off-hours mass access,  $\geq 50$  MB USB,  $\geq 200$  MB upload,  $\geq 15$  cross-department accesses, the leaver spike). Per-user score =  $0.7 \cdot \text{worst\_day} + 0.3 \cdot \text{average}$ .

## Results

From results/risk\_scores.csv. Perfect precision/AUC reflects cleanly-separable synthetic data — a pipeline sanity check, not a real-world accuracy claim.

| Metric                | Value                                |
|-----------------------|--------------------------------------|
| Events / users / days | 25,204 / 60 / 30                     |
| Feature rows          | 1,361 user-days                      |
| Precision@4           | 1.00 (4/4 insiders in top 4)         |
| ROC-AUC               | 1.000                                |
| Score separation      | insiders 49.5-90.1; top benign ~37.6 |

## Risk-score components

| Component       | Weight | Method                                          |
|-----------------|--------|-------------------------------------------------|
| rules layer     | 0.45   | analyst known-bad patterns                      |
| IsolationForest | 0.35   | 200-tree anomaly rank                           |
| statistical     | 0.20   | self + peer z-scores                            |
| per-user        | --     | $0.7 \cdot \text{worst} + 0.3 \cdot \text{avg}$ |

## Verification

Re-ran here, 9/9 (~8 s). test\_peer\_group\_deviation asserts the after\_hours\_exfil insider sits  $>2$  sigma above department peers.

19

Cyber Security  
Codec Technologies India  
Internship 2026

## Insider Threat Detection

User Behaviour Analytics: stats + ML + rules

### AT A GLANCE

25,204

Events

60 / 30

Users / days

1.00

Precision@4

1.000

ROC-AUC

4 / 4 top-4

Insiders caught

9 / 9

Tests re-run

synthetic (CERT-like)

Data

### VERDICT

TESTS 9/9

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

Eswar Mahalingam

Cyber Security Intern  
Ghaziabad, UP, India  
+91-9360548243  
eswarmba05313@gmail.com  
linkedin.com/in/eswar-mahalingam

| Test group    | Checks                          | Result |
|---------------|---------------------------------|--------|
| Pipeline      | features build, scores in range | PASS   |
| Detection     | all 4 insiders ranked top-4     | PASS   |
| Peer analysis | insider >2 sigma above peers    | PASS   |
| Rules         | known-bad patterns fire         | PASS   |
| Total         | 9 tests                         | 9/9    |

### How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test\_\*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp\_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

| Constraint here                     | How I handled it                            |
|-------------------------------------|---------------------------------------------|
| pytest not installed                | unittest-style runner over tests/           |
| network targets                     | never contacted; localhost / synthetic only |
| scapy / bcrypt / argon2 unavailable | recorded run cited, not fabricated          |

### Real-tool mapping, limitations & next steps

The blended detector is real; in a SOC it would run over a SIEM/UEBA stack at scale.

| Ran here                        | Real stack           | Note                           |
|---------------------------------|----------------------|--------------------------------|
| stats + IsolationForest + rules | ELK / Splunk UBA     | same blend at scale            |
| synthetic CERT-like logs        | CMU CERT r1-r6.2     | swap generate_data.py          |
| daily aggregates                | LSTM sequence models | temporal modelling future work |

### Ethics, data integrity & declared substitutions

Educational and defensive. **Data integrity:** all activity logs are synthetic and generated by the project, inspired by the structure of the CMU CERT Insider Threat dataset (same event families) but with no real people or events. **Declared substitution:** the project does not download CERT; swap generate\_data.py for a CERT loader with the same columns to run on real data. Insider monitoring has serious privacy/legal implications and must be governed by policy, notice and proportionality — I state this explicitly.

### Learnings

Blending three signal types — statistical, unsupervised, rules — taught me that no single method catches every insider pattern (the quiet leaver vs the loud exfiltrator). Being upfront that perfect scores come from separable synthetic data is essential honesty: the pipeline works, but real logs are far noisier.

### Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Perfect scores come from synthetic, cleanly-separable data — a sanity check, not a real-world claim.
- Rules thresholds are tuned to this generator; real deployments need continuous tuning.
- No temporal/sequence modelling; features are daily aggregates.
- Ground-truth labels exist because we planted them; real UBA is mostly unsupervised.
- Risk = 0.45\*rules + 0.35\*IsolationForest + 0.20\*statistical, scaled 0-100.
- 25,204 events, 60 users, 30 days, 1,361 user-day feature rows.
- Precision@4 = 1.00 and ROC-AUC = 1.000 on this dataset.
- Insider monitoring carries privacy/legal duties: policy, notice, proportionality.
- 25,204 synthetic events, 60 users, 30 days, 1,361 feature rows.
- Precision@4 = 1.00 (all 4 seeded insiders in the top 4).
- ROC-AUC = 1.000 on per-user risk.
- Insiders scored 49.5-90.1; highest benign ~37.6.
- after\_hours\_exfil insider sits >2 sigma above department peers.
- Four planted scenarios: after-hours exfil, big upload, unusual access, leaver.
- Data is CERT-inspired but fully synthetic; no real person profiled.
- References: CMU CERT dataset, IsolationForest (ICDM 2008), MITRE TA0010.
- Reference: CMU CERT Insider Threat Test Dataset.
- Reference: scikit-learn IsolationForest; Liu et al. (ICDM 2008).
- Reference: MITRE ATT&CK; Exfiltration (TA0010); Elastic SIEM; NIST 800-53 AU.
- risk\_scores.csv records Precision@4 = 1.00, ROC-AUC = 1.000.
- Data is CERT-inspired but fully synthetic; no real person profiled.